

- WS2812b LED - GP21
- All ground wires to the circuit ground plane
- All +5 Volt pins to GP40 Vbus pin

Step 3: Install CircuitPython

Head to this website (https://circuitpython.org/board/raspberry_pi_pico/) and download the latest stable release for CircuitPython. We used version 7.3.3. This is a firmware flash, so you have to be a little careful. Take your Raspberry Pi board (we used a Pi Pico) and press the BOOTSEL button and hold it. Then connect the USB Cable to the Pi. If you're not using a Raspberry Pi Pico, then you will have to download the appropriate release for your Pi board. You can find

by including the following code at the beginning.

CODE:

```
from adafruit_circuitplayground.express import cpx
```

You can use the following code sample to push Capital letter A on one button and Ctrl+X on another button.

CODE:

```
import usb_hid
from adafruit_circuitplayground.express import cpx
from adafruit_hid.keyboard import Keyboard
from adafruit_hid.keycode import Keycode
```

Now the question is, how do you decide which button is which?

Step 6: Identify the buttons

Because we've been careful about labeling the buttons while designing the circuit and are also aware of which GPIO pins the rows and columns are connected to. It's quite easy.

Define Rows and columns. And then also map the keys.

CODE:

```
cols = [digitalio.
DigitalInOut(x) for x in
(board.D0, board.D1, board.
D2, board.D3)]
rows = [digitalio.
DigitalInOut(x) for x in
(board.D4, board.D5, board.
D6, board.D7, board.D8)]
keys = ((a, b, c, d),
        (e, f, g, h),
        (i, j, k, l),
        (m, n, o, p))
```

And then initialise the keyboard matrix.

CODE:

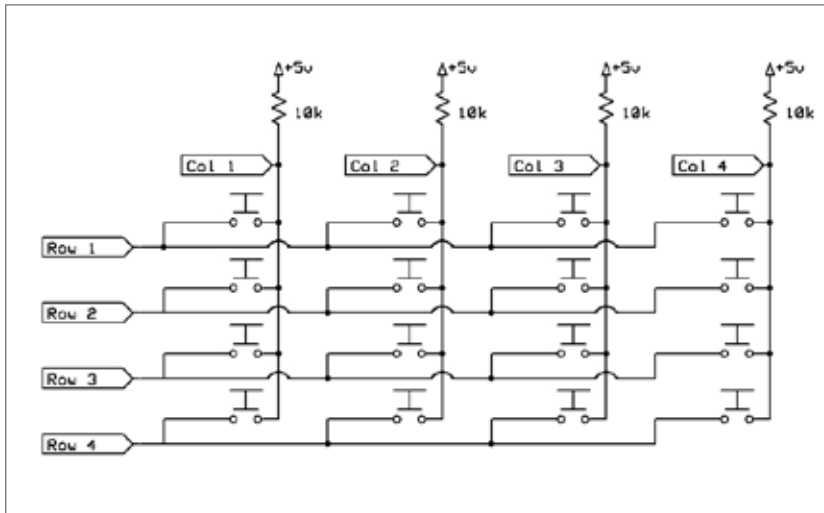
```
keypad = adafruit_matrixkeypad.Matrix_Keypad(rows,
cols, keys)
```

And once that's done. You can send the keystrokes by constantly reading all the inputs to check if any button is pressed. Then the appropriate key will be printed out.

CODE:

```
while True:
    keys = keypad.pressed_keys
    if keys:
        print("Pressed: ",
keys)
        time.sleep(0.1)
```

We're done with mapping all the keyboard switches. That's about it for Part-1 of this tutorial. In Part-2, we will be dealing with the rotary encoders. Let us know if Raspberry Pi projects are right up your alley, and if there are any project ideas that you might have. We'd certainly try to cook up some good DIY projects in the months to come. **1**



Wiring diagram for Keypad matrix

the release for Raspberry Pi 4B board here (https://circuitpython.org/blinky/raspberry_pi_4b/).

Once you've connected the Raspberry Pi board to the PC with the BOOTSEL button pressed, you should be able to see it as a storage device. Drag and drop the CircuitPython file on the Raspberry Pi. The board will restart and then show up as Circuitpy. This means that the installation was proper.

Step 4: Get an IDE

You can either use MU IDE (<https://codewith.mu/>) or you can use Thonny IDE (<https://thonny.org/>). Both will work properly with this tutorial. Install the IDE. We used MU IDE.

Step 5: Time to Code

You can call the CircuitPython library

```
kbd = Keyboard(usb_hid.
devices)
```

```
while True:
    if cpx.button_a:
        kbd.send(Keycode.
SHIFT, Keycode.A) # Type
capital 'A'
        while cpx.button_a:
            # Wait for button to be
            released
        pass
```

```
if cpx.button_b:
    kbd.send(Keycode.
CONTROL, Keycode.X) #
control-X key
    while cpx.button_b:
        # Wait for button to be
        released
    pass
```